AdaptiveLogSoftmaxWithLoss#

https://pytorch.org/docs/stable/generated/torch.nn.modules.adaptive.Adaptive

Description:

Efficient softmax approximation. As described in Efficient softmax approximation for GPUs by Edouard Grave, Armand Joulin, Moustapha Cissé, David Grangier, and Hervé Jégou. Adaptive softmax is an approximate strategy for training models with large output spaces. It is most effective when the label distribution is highly imbalanced, for example in natural language modelling, where the word frequency distribution approximately follows the Zipf's law. Adaptive softmax partitions the labels into several clusters, according to their frequency. These clusters may contain different number of targets each. Additionally, clusters containing less frequent labels assign lower dimensional embeddings to those labels, which speeds up the computation. For each minibatch, only clusters for which at least one target is present are evaluated. The idea is that the clusters which are accessed frequently (like the first one, containing most frequent labels), should also be cheap to compute – that is, contain a small number of assigned labels.

Function Signature

```
class
torch.nn.modules.adaptive.AdaptiveLogSoftmaxWithLoss(in_feat
ures, n_classes, cutoffs, div_value=4.0, head_bias=False,
device=None, dtype=None)[source]#

Parameters

Returns

Return type

Shape:
```

Returns:

output (Tensor)

Notes & Warnings

⚠ WARNING Warning Labels passed as inputs to this module should be sorted according to their frequency. This means that the most frequent label should be represented by the index 0, and the least frequent label should be represented by the index n_classes - 1.

NOTE Note This module returns a NamedTuple with output and loss fields. See further documentation for details.

NOTE Note To compute log-probabilities for all classes, the log_prob method can be used.

Detailed Documentation

Documentation

Efficient softmax approximation.

As described in Efficient softmax approximation for GPUs by Edouard Grave, Armand Joulin, Moustapha Cissé, David Grangier, and Hervé Jégou.

Adaptive softmax is an approximate strategy for training models with large output spaces. It is most effective when the label distribution is highly imbalanced, for example

in natural language modelling, where the word frequency distribution approximately follows the Zipf's law.

Adaptive softmax partitions the labels into several clusters, according to their frequency. These clusters may contain different number of targets each. Additionally, clusters containing less frequent labels assign lower dimensional embeddings to those labels, which speeds up the computation. For each minibatch, only clusters for which at least one target is present are evaluated.

The idea is that the clusters which are accessed frequently (like the first one, containing most frequent labels), should also be cheap to compute – that is, contain a small number of assigned labels.

We highly recommend taking a look at the original paper for more details.

cutoffs should be an ordered Sequence of integers sorted in the increasing order. It controls number of clusters and the partitioning of targets into clusters. For example setting cutoffs = [10, 100, 1000] means that first 10 targets will be assigned to the 'head' of the adaptive softmax, targets 11, 12, …, 100 will be assigned to the first cluster, and targets 101, 102, …, 1000 will be assigned to the second cluster, while targets 1001, 1002, …, n_classes - 1 will be assigned to the last, third cluster.

div_value is used to compute the size of each additional cluster, which is given as ⌊in_featuresdiv_valueidx⌋\left\lfloor\frac{\texttt{in_features}}{\texttt{div_value}^{idx}}\right\rfloor⌊div_valueidxin_features⌋, where idxidxidx is the cluster index (with clusters for less frequent words having larger indices, and indices starting from 111).

head_bias if set to True, adds a bias term to the 'head' of the adaptive softmax. See paper for details. Set to False in the official implementation.

Labels passed as inputs to this module should be sorted according to their frequency.

This means that the most frequent label should be represented by the index 0, and the least frequent label should be represented by the index n_classes - 1.

This module returns a NamedTuple with output and loss fields. See further documentation for details.

To compute log-probabilities for all classes, the log_prob method can be used.

in_features (int) – Number of features in the input tensor

n_classes (int) – Number of classes in the dataset

cutoffs (Sequence) – Cutoffs used to assign targets to their buckets

div_value (float, optional) – value used as an exponent to compute sizes of the clusters. Default: 4.0

head_bias (bool, optional) – If True, adds a bias term to the ‘head’ of the adaptive softmax. Default: False

output is a Tensor of size N containing computed target log probabilities for each example

loss is a Scalar representing the computed negative log likelihood loss

NamedTuple with output and loss fields

input: (N,in_features)(N, \texttt{in_features})(N,in_features) or (in_features) (\texttt{in_features})(in_features)

target: (N)(N)(N) or ()()() where each value satisfies 0<=target[i]<=n_classes0 <= \texttt{target[i]} <= \texttt{n_classes}0<=target[i]<=n_classes

output1: (N)(N)(N) or ()()()

Runs the forward pass.

Compute log probabilities for all n_classes\texttt{n_classes}n_classes.

input (Tensor) – a minibatch of examples

log-probabilities of for each class ccc in range 0<=c<=n_classes0 <= c <= \texttt{n_classes}0<=c<=n_classes, where n_classes\texttt{n_classes}n_classes is a parameter passed to AdaptiveLogSoftmaxWithLoss constructor.

Input: (N,in_features)(N, \texttt{in_features})(N,in_features)

Output: (N,n_classes)(N, \texttt{n_classes})(N,n_classes)

Return the class with the highest probability for each example in the input minibatch.

This is equivalent to self.log_prob(input).argmax(dim=1), but is more efficient in some cases.

input (Tensor) – a minibatch of examples

a class with the highest probability for each example

Input: (N,in_features)(N, \texttt{in_features})(N,in_features)

Resets parameters based on their initialization used in __init__.